A Constraint-Based CKY Parser Implementation for English

Tarık Emre Tıraş and **Mehmet Ali Özdemir**
Department of Linguistics / Computer Engineering
Boğaziçi University, Istanbul, Turkey
tarik.tiras@std.bogazici.edu.tr malizdmr011@gmail.com

Abstract

Syntactic parsing is a fundamental component of Natural Language Processing pipelines, yet standard Context-Free Grammars (CFGs) often suffer from over-generation due to a lack of fine-grained linguistic constraints. This paper presents the implementation of a feature-based Cocke-Younger-Kasami (CKY) parser for English. Unlike standard probabilistic parsers, our system relies on a symbolic constraint satisfaction approach. It integrates a morphological analyzer to handle feature percolation (number, person, tense) and uses a unification-based grammar to strictly enforce subject-verb agreement, subcategorization frames, and auxiliary inversion rules. We describe the pipeline of normalizing the grammar into Chomsky Normal Form (CNF), the implementation of the bottom-up chart parsing algorithm, and the constraint validation logic. Evaluation on a designated test suite demonstrates that the system achieves high precision, successfully parsing complex valid structures while correctly rejecting 100% of ungrammatical inputs.

1 Introduction

Since their formalization by (Chomsky 1957), Context-Free Grammars (CFGs) have served as the standard backbone for modeling syntactic hierarchy. However, atomic CFGs are known to lack the expressive power required to capture complex morphosyntactic dependencies—such as number agreement and subcategorization—without suffering from a combinatorial explosion in the symbol set (Shieber 1986). For example, ensuring Subject-Verb Agreement using only atomic symbols would require distinct rules for singular and plural noun phrases ($\texttt{NP}_{sg}$, $\texttt{NP}_{pl}$), essentially duplicating the grammar. To address these limitations, we adopt a unification-based approach (Shieber 1986; Knight 1989), implementing a parser that augments standard CFG backbones with feature structures to enforce local constraints. This system decouples the structural rules from the morphosyntactic constraints. The core of our system is the Cocke-Younger-Kasami (CKY) algorithm, a polynomial-time dynamic programming approach for parsing context-free languages (Kasami 1965; Younger 1967). We extend standard CKY to support feature unification: a derivation is only valid if the morphosyntactic features of the child nodes satisfy specific constraint functions defined in the grammar. This paper details the system architecture, including the morphological analyzer, the grammar normalization pipeline (conversion to Chomsky Normal Form), the constraint enforcement logic, and the parsing algorithm itself.

2 MorphAnalyzer Class

The `MorphAnalyzer` class bridges raw text input and syntactic parsing by enriching terminal symbols with linguistic features (e.g., number, tense, case). The lexicon is stored as a dictionary mapping normalized words to lists of *(POS_tag, features_dict)* tuples.

For example, the unambiguous word *"booked"* is stored as `[("VBD", {tense: "past", subcat: "tr"})]`. Ambiguous words like *"book"* map to multiple entries: a singular noun (`NN`, `{num: "sg", per: 3}`) and a base transitive verb (`VB`, `{tense: "base", subcat: "tr"}`). The `feed_to_grammar` method iterates through these entries to generate terminal grammar rules (e.g., `NN` $\rightarrow$ `"book"`).

2.1 Lexicon Population

The `build_lexicon` method preloads a curated lexicon with English words across categories with their POS tags. We adopted the standard Penn Treebank POS tagset (Marcus, Santorini, and Marcinkiewicz 1993) for all lexical categories.

Nouns. Nouns are divided into two categories based on *number* feature: Singular Nouns (`NN`), Plural Nouns (`NNS`).

Pronouns. In contrast to nouns, pronouns have a lot more distinctive features, as illustrated in Table 1. Personal pronouns and object pronouns share the same POS Tag, although they are separated with *accusative* feature. Possessive pronouns use the *PRP$* tag. On the other hand, their further morphological features are not encoded since they do not have any role in syntactic constraints.

Table 1: Examples of Pronoun Classification

Pronoun	POS Tag	Number	Person	Case
i	PRP	singular	1	nominative
we	PRP	plural	1	nominative
you	PRP	*	2	nominative
him	PRP	singular	3	accusative
us	PRP	plural	1	accusative
my	PRP$	–	–	–

Verbs. They are categorized by tense, number, person, and subcategorization. Below is a summary of the four primary verb forms:

Table 2: Verb Categories and Features

Form	POS Tag	Tense	Number	Person	Subcategorization
Past	VBD	past	sg/pl	–	transitive (tr), intransitive (intr), copulative (cop), auxiliary (aux)
Present 3sg	VBZ	pres	sg	3	transitive (tr), intransitive (intr), copulative (cop), auxiliary (aux)
Present pl	VBP	pres	pl	–	transitive (tr), intransitive (intr), auxiliary (aux)
Base	VB	base	–	–	transitive (tr), intransitive (intr)

By specifying how verbs combine with other elements, **subcategorization features** enforce syntactic constraints:

- `tr`: Requires a direct object (e.g., *"buy a book"*).
- `intr`: Forbids a direct object (e.g., *"go home"*).
- `cop`: Links subjects to complements (e.g., *"is old"*).
- `aux`: Used in questions/negation (e.g., *"does eat"*).

Prepositions. They are categorized by their semantic role using a type feature to guide syntactic analysis. This classification helps resolve attachment ambiguities and ensures proper phrase structure validation.

- **Directional (dir):** Indicate movement or direction (e.g., *to* (as TO or IN), *from*).
- **Locative (loc)**: Specify location or position (e.g., *in*, *under*, *at*).
- **General (gen)**: Express abstract relationships, such as purpose and accompaniment (e.g., *for*, *with*, *about*).

Adjectives, Adverbs, and Other POS Tags. Adjectives (JJ) describe nouns, enriching phrases like *"historical book"* or *"summer nights"*, where "summer" is tagged as JJ to facilitate compound noun handling. Adverbs (RB) modify verbs, adjectives, or other adverbs, expressing manner, time, or degree *(e.g., "very loud"* (degree) or *"yesterday" (time)*). The Wh-adverb (WRB) "when" introduces questions, while the superlative adverb (RBS) "most" intensifies adjectives *(e.g., "most beautiful")*. Modals (MD) like *"will"* and *"can"* combine with base verbs to express possibility or necessity, as in *"will go"*. Conjunctions (CC) such as *"and"* link words or clauses. These tags, assigned by the MorphAnalyzer, ensure accurate syntactic analysis and constraint enforcement during parsing.

2.2 Lexicon Access

The get_tags method retrieves all part-of-speech (POS) tags and associated feature dictionaries for a given word, normalized to lowercase. For example, get_tags("book") returns [("NN", "num": "sg", "per": 3), ("VB", "tense": "base", "subcat": "tr")], reflecting its dual noun/verb usage. Unknown words return an empty list, ensuring graceful degradation.

2.3 Grammar Integration

The feed_to_grammar method integrates the lexicon into the parsing system by converting each lexical entry into a terminal grammar rule. For instance, the entry for *"book"* as a noun (NN) generates the rule NN → "book" with features num: "sg", per: 3.

3 Grammar Representation and Normalization

The structural backbone of the parser is the Grammar class, which manages the storage and retrieval of production rules. Unlike a standard CFG, where rules are simple mappings of Left-Hand Side (LHS) to Right-Hand Side (RHS), our implementation associates every rule with two additional components:

1. **Validator:** A callable function (e.g., agree_subj_verb) that accepts child nodes and returns a unified feature dictionary or None if the derivation is invalid.
2. **Static Features:** Fixed features assigned to the LHS (e.g., assigning type: 'q_wh' to a question structure).

To optimize the parsing process, rules are indexed in two hash maps: rules_by_rhs for bottom-up lookups during the CKY phase, and rules_by_lhs for top-down traversals and debugging.

3.1 Conversion to Chomsky Normal Form (CNF)

A prerequisite for the standard CKY algorithm is that the grammar must be converted into Chomsky Normal Form (CNF), restricting productions to strict binary (A → B C) or unary-terminal (A → w) formats. Natural English grammar, however, often contains mixed rules (e.g., NP → DT book) or n-ary productions (VP → VBD NP PP). To handle this, we implemented a two-stage normalization pipeline:

1. **Terminal Separation (`convert_case1`):** Mixed rules are split by introducing dummy non-terminals. A rule like NP → DT "book" becomes X_BOOK → "book" and NP → DT X_BOOK.
2. **Binarization (`convert_case3`):** N-ary productions are decomposed recursively. A ternary rule S → A B C becomes X_A_B → A B and S → X_A_B C. Feature validators are attached to the top-most rule of the chain to preserve constraint logic.

4 Morphosyntactic Constraints and Validator Functions

We employ a suite of constraint functions to enforce grammatical rules during parsing. These functions validate feature compatibility between nodes, ensuring syntactic well-formedness.

Subject-Verb Agreement (`agree_subj_verb()`). Enforces subject-verb agreement for the S → NP VP production. It validates the unification of number and person features between the NP and VP, handling the English exception where the 1st-person singular ("*I*") governs plural verb forms. It rejects

VPs headed by base-form verbs in root declarative clauses (e.g., rejecting "*She tell a story*"), while permitting them in imperative or modal contexts.

Determiner-Noun Agreement `(agree_det_noun).` Validates number consistency within Determiner Phrases (NP → DT NN). It enforces unification between the determiner's number feature and the head noun, pruning mismatched pairs such as "*these book*" or "*a students*".

Subcategorization `(require_transitive, require_intransitive).` Enforces strict argument structure constraints based on the verbal head's subcategorization features.

- **Transitive Frames:** Ensures verbs marked [subcat: tr] are followed by an NP object (e.g., *bought a book*), rejecting intransitive usages (e.g., *I went the school*).
- **Intransitive Frames:** Ensures verbs marked [subcat: intr] appear without direct objects (e.g., *sleep*), or with appropriate directional/locative PP adjuncts (e.g., *went to the park*).

Copular Predication `(require_copula).` Restricts the distribution of predicative adjectives. This function validates that Adjective Phrases (AP) or nominal predicates only attach to verbs carrying the [subcat: cop] feature (e.g., *is, was*), thereby blocking ungrammatical modification of lexical verbs (e.g., *The music read beautiful*).

Interrogative Structure and Finiteness `(check_aux_question, agree_question_body).` These functions model Subject-Auxiliary Inversion and do-support in Yes/No questions.

- **Auxiliary Check:** Verifies that root interrogatives begin with a valid auxiliary operator (e.g., *Did, Can*), distinguishing them from declarative word orders.
- **Non-Finite Body:** Enforces a base-form constraint on the lexical verb within the question body. For example, in "*Did he go?*", it validates that the main verb go is non-finite, rejecting double-tense marking (e.g., *Did he went?*).

Feature Percolation `(pass_head_features).` Implements X-bar projection principles, propagating morphosyntactic features (tense, number, person) from the head child to the parent node. This ensures that higher-level rules (e.g., S → NP VP) have access to the features of constituents (e.g., the plurality of an NP head) determined at lower levels of the tree.

Semantic Selection `(check_time_adjunct).` Constrains the modification of Verb Phrases by Noun Phrases (VP → VP NP). This function acts as a semantic filter, verifying that the adjoined NP carries the feature [sem: time]. This distinguishes temporal adjuncts (e.g., *slept yesterday*) from invalid direct objects in intransitive contexts (e.g., *slept the book*).

Rule-Specific Constraints (Lambda Functions). For idiosyncratic constructions, we utilized anonymous functions directly within the grammar definition to enforce immediate constraints:

- **Imperatives:** The S → VP rule enforces a base-form constraint ([tense: base]) on the head verb (e.g., *Go home!*).
- **Modals:** The VP → MD VP rule propagates a [tense: fut] feature while enforcing that the complement VP is non-finite.
- **Negation:** The auxiliary construction for negative imperatives (VP → VB RB) strictly verifies the lexical presence of *"do not"*.

5 Syntactic Production Rules

The core grammar is implemented as a Feature-Based Context-Free Grammar (CFG), adapting standard English phrase structure rules (Carnie 2021) with constraint augmentation. Rather than over-generating with a broad set of permissive rules, we structured the productions to strictly enforce the subcategorization requirements validated by the constraint functions described in Section 4.

Clause Structure and Mood. The backbone of the grammar rests on the declarative rule `S → NP VP`, which serves as the domain for subject-verb agreement. To handle distinct moods, we introduced specialized root productions:

- **Imperatives:** Modeled via the unary production `S → VP`, which is constrained to accept only base-form verbs (e.g., *"Do not go"*).
- **Interrogatives:** Yes/No questions are constructed via Subject-Auxiliary Inversion rules, such as `S → Aux S_body`. The intermediate non-terminal `S_body` represents the un-inverted subject-predicate pair (`NP VP_base`), isolating the finiteness check to the auxiliary operator.
- **Wh-Movement:** Wh-questions are derived through a dedicated head structure `S_Qhead → Wh Aux`, which combines with the sentence body (e.g., *"When will"* + *"the teacher come"*).

Verb Phrase (VP) Internal Structure. Unlike standard CFGs that might use a generic `VP → V (NP) (PP)` rule, our grammar explicitly separates `VP`s based on valency to facilitate constraint checking:

- **Transitive:** `VP → V_tr NP`
- **Intransitive:** `VP → V_intr` and `VP → V_intr PP`
- **Copular:** `VP → V_cop AdjP` and `VP → V_cop NP`
- **Modal Projections:** Future and modal constructions are handled via `VP → MD V`, where the modal head dictates the tense features of the parent node while enforcing a non-finite constraint on the complement `VP`.

Noun Phrase (NP) Recursion and Modification. The grammar supports complex nominal structures through recursive production rules. Beyond standard determiner-noun pairings, we implemented:

- **Recursive Adjunction:** The rule `NP_adj → JJ NP_adj` allows for stacked adjectival modification (e.g., *"warm summer nights"*).
- **Post-Nominal Modification:** Rules such as `NP → NP PP` allow for prepositional attachment, handled via right-branching structures to preserve feature percolation from the head noun.

6 The CKY Parsing Algorithm

We implemented the Cocke-Younger-Kasami (CKY) algorithm, a generic bottom-up chart parsing method. The algorithm constructs a triangular matrix (chart), where each cell $[i][j]$ contains a list of valid constituents spanning the word indices from i to j.

Chart Initialization. The process begins by populating the diagonal of the chart. For every word w at index i, the system queries the `Grammar` for terminal rules producing w. These are instantiated as nodes containing the symbol and the features retrieved from the `MorphAnalyzer`.

The Main Loop and Unification. The algorithm iterates through span lengths l from 1 to n (sentence length), and split points k. For every potential binary combination of a left child (span $[i][k]$) and a right child (span $[k][j]$), the system performs the following checks:

1. **Rule Existence:** It checks if a grammar rule exists for the pair of symbols (*Left.symbol, Right.symbol*).
2. **Constraint Validation:** If a rule exists, its associated validator function is executed against the specific feature dictionaries of the two child nodes.

If the validator returns a valid feature set, a new node is added to cell $[i][j]$. This step effectively prunes the search space; syntactic trees that violate agreement or subcategorization constraints are discarded immediately during chart construction, rather than being filtered later.

Handling Unary Chains. Standard CKY does not natively handle unary non-terminal productions (e.g., `NP → NN`) inside the main loop. We addressed this by implementing a `while added_new` loop within every cell. After binary combinations are processed, the system iteratively checks if any newly created node can be promoted via a unary rule, allowing for chains like `NP → NN → "book"`.

Tree Reconstruction. To recover the parse tree from the chart, we maintain a backpointer table mapping every node to its children. Once the chart is filled, we check cell $[0][n]$ for a start symbol `S`. If found, the `get_parse_tree` function recursively traverses the backpointers. Finally, a `unbinarize` utility function flattens the artificial intermediate rules created during CNF conversion, restoring the original n-ary structure of the grammar for readability.

7 Evaluation

To assess the coverage and precision of the grammar, we constructed a test suite of 30 sentences designed to target specific morphosyntactic phenomena, including subject-verb agreement, argument structure (valency), and wh-movement.

7.1 Experimental Setup

The test suite consists of two subsets:

- **Valid Set (20 sentences):** Grammatically correct sentences containing transitive/intransitive verbs, recursive adjectives, and interrogative structures. Success is defined as generating at least one valid parse tree.
- **Invalid Set (10 sentences):** Constructed ungrammatical sentences violating specific constraints (e.g., number agreement, word order). Success is defined as the parser returning *None* (correct rejection).

7.2 Quantitative Results

The parser demonstrated high precision, correctly rejecting 100% of the 10 ungrammatical inputs. On the valid set of 20 sentences, it achieved 95% coverage, failing on only one instance due to a lexical gap. Overall accuracy on the test suite was 96.7% (29/30).

7.3 Qualitative Analysis

The results confirm that the constraint functions described in Section 4 effectively enforce syntactic rules.

Agreement and Morphology. The system correctly parsed sentences requiring feature unification, such as *"Every student passed the difficult exam."* Conversely, it successfully rejected subject-verb number mismatches like *"My friend live in a village"* and determiner-noun mismatches like *"Every students passed."* This validates the effectiveness of the `agree_subj_verb` and `agree_det_noun` constraints.

Subcategorization. Verbal valency was strictly enforced. Transitive usages like *"She tells a long story"* were parsed correctly, while invalid argument structures, such as *"The teacher read to the students a book"* (violating the dative construction rules) and *"She tell a story"* (base form in a root declarative), were rejected.

Complex Structures. The parser successfully handled non-canonical word orders, including Subject-Auxiliary Inversion in questions. For example, the sentence *"When will the new teacher come here"* was correctly analyzed as an `S_Q_HEAD` structure, linking the auxiliary *will* to the subject while enforcing the base form on the main verb *come*.

8 Conclusion

In this paper, we presented a comprehensive implementation of a constraint-based CKY parser for English. By integrating a morphological lexicon with a feature-unifying grammar, we demonstrated that it is possible to enforce strict syntactic correctness—including subject-verb agreement, argument structure valency, and auxiliary support—without resorting to the massive symbol redundancy typical of atomic CFGs.

The system's modular separation of grammar and constraint logic facilitates extensibility. Future work includes integrating probabilistic weights (PCFG) for ambiguity resolution and expanding lexical coverage. Our results confirm that lightweight, rule-based systems remain a viable and transparent method for enforcing grammatical precision.

References

Carnie, Andrew (2021). *Syntax: A Generative Introduction*. 4th ed. Hoboken, NJ: Wiley-Blackwell.

Chomsky, Noam (1957). *Syntactic Structures*. Mouton.

Kasami, Tadao (1965). "An Efficient Recognition and Syntax-Analysis Algorithm for Context-Free Languages". In: URL: https://api.semanticscholar.org/CorpusID:61491815.

Knight, Kevin (Mar. 1989). "Unification: a multidisciplinary survey". In: *ACM Comput. Surv.* 21.1, pp. 93–124. ISSN: 0360-0300. DOI: 10.1145/62029.62030. URL: https://doi.org/10.1145/62029.62030.

Marcus, Mitchell P., Beatrice Santorini, and Mary Ann Marcinkiewicz (1993). "Building a Large Annotated Corpus of English: The Penn Treebank". In: *Computational Linguistics* 19.2. Ed. by Julia Hirschberg, pp. 313–330. URL: https://aclanthology.org/J93-2004/.

Shieber, Stuart M. (1986). "An Introduction to Unification-Based Approaches to Grammar". In: *CSLI Lecture Notes*. URL: https://api.semanticscholar.org/CorpusID:222273301.

Younger, Daniel H. (1967). "Recognition and parsing of context-free languages in time n3". In: *Information and Control* 10.2, pp. 189–208. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(67)80007-X. URL: https://www.sciencedirect.com/science/article/pii/S001999586780007X.